

Shell Scripting Lab

- part 1: the shell
 - the prompt:
 - different styles of prompts
 - can give different information like current working directory, username and machine name
 - also something that is configurable
 - shell variables \$PS1-\$PS4 can be used to configure the prompt
 - **Exercise:** check your prompt and see what information it displays. Write down what information you would like displayed (we will use this later).
 - locating shell programs
 - **Exercise:** find the locations of bash, tcsh, csh and sh.

Shell Scripting Lab

- locating the files that the shell loads
 - the shell goes through a series of files when it starts up
 - some are system settings, some are user settings
 - system-wide configurations can be in different places, but user-defined settings are only in one place
 - **Exercise:** find the location of the file that users can use to put environment settings to be loaded on logins. Open the file in your favorite editor and look around. What do the lines (if any) tell you about the process of loading environment settings?
- different environments for doing shell work
 - **Exercise:** use the echo command on the command line to write text to the screen. Try creating a file with that same command in it and run with **bash <myfile>**.

Shell Scripting Lab

- part 2: the command line
 - parts of a command - running with and without options and arguments:
 - **Exercise:** run the following and note the result. Check the manual pages to see what each option means.
 - `ls`
 - `ls -la`
 - `ls -lrth /usr/local`
 - debugging command errors:
 - **Exercise:** run the following commands. What was the error message and how would you fix it?
 - `ls -l a`
 - `cd workdir`
 - `echo "echo $HOME" > myscript.sh; myscript.sh`

Shell Scripting Lab

- part 3: a first script
 - a simple task on the command line
 - **Exercise:** run the following on the command line:
 - `MYVAR="`which gnome-terminal`"; echo $MYVAR`
 - change `gnome-terminal` to another command
 - that same task in a script (or the basic parts of a script):
 - **Exercise:**
 - write up a script that contains the above command
 - run it using `bash <myscript>`
 - add the line below as the very first line to that script:
`#!/bin/bash`
 - run the command using `./<my_script>`. why won't it run?
 - Give examples of unsuitable script names and the reason for your choice.

Shell Scripting Lab

- part 4: setting and using variables
 - setting variables for a session:
 - **Exercise:** set the following variables to some value:
 - MYVAR
 - SOMEJUNK
 - ABCD
 - **Exercise:** type "bash" and look at the results of the above assignments - why do you see the results you do? How would you fix that?
 - setting variables more permanently
 - **Exercise:** set the above variables to linger for multiple login sessions
 - **Exercise:** echo to the screen the results of the variable assignments. Check pre-set variables with the **env** command.

Shell Scripting Lab

- part 5: setting variables as results of other variables and commands
 - introduction to quotes:
 - different quotes have different meaning in bash:
 - ` (the back tick): execute the command
 - ' (single quote): treat string literally
 - " (double quote): expand any expressions within
 - **Exercise:** do the following and comment on what you see
 - MYVAR=junk
 - echo 'variable MYVAR has \$MYVAR'
 - echo "variable MYVAR has \$MYVAR"
 - variables from other variables and command output:
 - **Exercise:** set 3 variables, named X1, X2 and X3, to some value. Then concatenate them and assign the result to another variable.
 - **Exercise:** store the results of `pwd` to a variable and print.

Shell Scripting Lab

- part 6: setting up your environment
 - setting PATH, LD_LIBRARY_PATH, MANPATH
 - **Exercise:** do the following to append a bin directory to the command search path (stored in PATH)
 - `mkdir ~/mybin`
 - `export PATH=$PATH:$HOME/mybin`
 - now, do the same two steps, but create a library directory and append it to LD_LIBRARY_PATH
 - do the same again for MANPATH
 - try the variable assignment in two steps
 - add the same commands to your `.bashrc` and run with `~/.bashrc`

Shell Scripting Lab

- command aliasing:

- **Exercise:** `alias` is a useful command to create shortcuts. Check the man page for `alias` and use the info there to alias the following commands:

- `ls -lrth`

- `rm -i`

- `ps -u $USER`

- try out a few for yourself

- changing your prompt:

- **Exercise:** check the man page for `bash` for all options for what to include in the prompt. Then try the following:

- `PS1="\u@\H: \W \>"; export PS1`

- `PS1="Hello \u. Your wish is my command!>"; export PS1`

- try one of your own.

Shell Scripting Lab

- part 7: if, switch
 - the command line parser revisited
 - **Exercise:** write a script to do some action based on options given.
 - testing expressions
 - **Exercise:** write if statements with the following types of tests:
 - arithmetic
 - strings
 - file/directory existence
 - set/unset variables
 - presence/absence of arguments to the script

Shell Scripting Lab

- part 8: while/until, for
 - reading a file line by line
 - **Exercise:** write a script to read in options from a file and echo them to the screen
 - logging into more than one machine:
 - **Exercise:** on the command line, write a for loop to log you in sequentially to the ACEnet machines
 - multiple job submissions in one script (CPMD example)
 - **Exercise:** multiple job submission
 - mainly to give you an example of file templating and job submission for multi-step jobs
 - this is but one of many possible examples...
 - we won't be running this script - modify it for your own work on a cluster

Shell Scripting Lab

- part 9: functions
 - function to echo its input
 - **Exercise:** write a function to take an option from the command line and echo it back
 - function to do something with the input
 - **Exercise:** write a function to take two strings as input and concatenate them into one, then print the result
 - try different string types and comment on the results

Shell Scripting Lab

- part 10: I/O
 - read from the command line in a script
 - **Exercise:** write a script to read something in from the command line and echo it back
 - **Exercise:** use the read statement to read a value in from the command line. The first time, have it echo what you type. Then change the read statement to hide what you write.
 - read from a file
 - **Exercise:**
 - write a file with some lines in it, all with the same number of fields.
 - write a script to read those lines and echo them back to the screen